



GamP-SCPeriop

Plataforma de gestão e gamificação de percursos de formação perioperatória

Sérgio Oliveira (n.º 15656)

Mestrado em Engenharia Informática
Escola Superior de Tecnologia
Instituto Politécnico do Cávado e do Ave

Resumo

[A completar: resumo em português (150 a 300 palavras): problema, solução, método e principais resultados.]

Palavras-chave: gamificação; formação perioperatória; gestão de percursos de aprendizagem; Blazor; ASP.NET Core.

Abstract

[A completar: abstract in English, translated from the Portuguese summary.]

Keywords: gamification; perioperative training; learning pathway management; Blazor; ASP.NET Core.

Índice

1 Introdução **1**

1.1 Problema . . . 1
1.2 Solução proposta . . . 1
1.3 Objetivos . . . 1
1.4 Âmbito . . . 2
1.5 Estrutura do documento . . . 2

2 Análise do problema **3**

2.1 Contexto e pressupostos . . . 3
2.2 Processos de negócio . . . 3
2.2.1 Preparar um percurso e inscrever os supervisionados . . . 3
2.2.2 Avaliar um supervisionado e atribuir badges . . . 5
2.3 Atores e perfis de acesso . . . 7
2.4 Requisitos . . . 7
2.4.1 Requisitos funcionais . . . 7
2.4.2 Requisitos não funcionais . . . 8

3 Utilizador e task model principal **11**

3.1 Utilizador principal . . . 11
3.2 Tarefa principal . . . 11

4 Site map **13**

4.1 Organização dos conteúdos . . . 13
4.2 Áreas e diferenças de acesso . . . 14
4.3 Ligações entre ecrãs . . . 14

5 Sistema visual **15**

5.1 Logótipo e identidade visual . . . 15
5.2 Paleta cromática . . . 15
5.2.1 Função das cores principais . . . 16
5.2.2 Contraste . . . 17
5.3 Tipografia . . . 17

5.4 Grelha, margens e espaçamento 17

6 Arquitetura **19**

6.1 Estilo arquitetural 19

6.2 Componentes 19

6.2.1 Pipeline do servidor 20

6.2.2 Serviços e lógica de negócio 21

6.3 Rotas 21

6.3.1 Rotas do cliente 22

6.3.2 Rotas da API 23

6.4 Autenticação e autorização 23

6.5 Catálogo de endpoints 24

6.6 Dados 27

7 Conclusão **29**

Lista de Figuras

2.1 Preparar um percurso e inscrever os supervisionados, em BPMN. 4
2.2 Avaliar um supervisionado e atribuir badges, em BPMN. 6

3.1 Task model da tarefa principal, em HTA. 12

4.1 Site map do GamP-SCPeriop. Cada caixa é um ecrã, com a rota por baixo; as setas cheias são a navegação principal e a tracejada um atalho. 13

6.1 Arquitetura do GamP-SCPeriop. A azul, os componentes do servidor; a seta tracejada é o descarregamento inicial da aplicação. 20
6.2 Resolução de um URL: primeiro no servidor, depois no navegador. 22
6.3 Sequência de pedidos ao gravar uma avaliação. 24

Lista de Tabelas

2.1 Pressupostos assumidos no desenho. 3
2.2 Estados de avaliação de uma componente. 5
2.3 Perfis de acesso. 7
2.4 Requisitos funcionais. 8
2.5 Requisitos não funcionais. 9

3.1 Perfil do utilizador principal: o supervisor. 11

4.1 Áreas da aplicação e respetivo acesso. 14

5.1 Paleta cromática da aplicação, com a função de cada cor. 16
5.2 Contraste das principais combinações de cor. 17
5.3 Utilização da tipografia. 17
5.4 Contentor principal por tamanho de ecrã. 18

6.1 Projetos da solução. 20
6.2 Controladores da API. 25
6.3 Catálogo de endpoints da API. 27

Siglas & Acrónimos

API Application Programming Interface (Interface de Programação de Aplicações). 2, 9, 19–21, 23, 24, 29

BPMN Business Process Model and Notation. 2, 3

HTA Hierarchical Task Analysis (Análise Hierárquica de Tarefas). 11

JWT JSON Web Token. 9, 20, 22–25

PDF Portable Document Format. 7, 8, 20, 25, 27

REST Representational State Transfer. 2, 9, 19, 20, 29

RGPD Regulamento Geral sobre a Proteção de Dados. 9

SPA Single-Page Application (Aplicação de Página Única). 2

WCAG Web Content Accessibility Guidelines. 8, 17

Glossário

Os termos deste glossário aparecem em itálico na primeira vez que são usados no texto.

badge Conquista atribuída automaticamente a um supervisionado quando atinge uma meta do percurso, por exemplo concluir um módulo. 1, 5

componente Tarefa teórica ou prática dentro de um módulo, associada a uma fase de aprendizagem e com um peso. Pode ter subcomponentes. 1

inscrição Ligação entre um supervisionado e um percurso. Ao inscrever, o sistema cria uma cópia própria dos módulos do percurso para esse aluno. 3

molde Estrutura-base de um percurso (módulos, componentes e badges), a partir da qual se criam percursos por clonagem. 1, 3

percurso Plano de estudo de um grupo de supervisionados, dividido em módulos e componentes, com datas e critérios de aprovação. Pertence a um supervisor. 1

supervisionado Estudante ou profissional em formação acompanhado por um supervisor num percurso. 1

1. Introdução

A formação em enfermagem perioperatória combina teoria com observação e prática em bloco operatório, ao longo de semanas e sob a orientação de um supervisor. Este documento apresenta a análise e o desenho do *GamP-SCPeriop*, uma aplicação web que organiza esses planos de formação e acrescenta-lhes um sistema de recompensas para motivar os estudantes.

1.1 Problema

Os planos de formação perioperatória são geridos, hoje, em documentos e folhas de cálculo dispersos. Cada supervisor monta o plano de cada estudante à mão, regista as avaliações em papel e calcula o progresso quando precisa dele. Deste processo resultam problemas recorrentes:

- Trabalho repetido: planos semelhantes são reescritos para cada grupo e para cada estudante;
- Falta de visibilidade: o supervisor não tem uma visão do progresso de todos os seus estudantes, e o estudante não sabe onde está no seu plano;
- Avaliação pouco estruturada: os critérios e as fases de aprendizagem variam de supervisor para supervisor;
- Pouca motivação: um plano longo, sem marcos visíveis, torna-se monótono para quem o percorre.

1.2 Solução proposta

O GamP-SCPeriop estrutura cada plano como um *percurso*, dividido em módulos e *componentes*, e associa cada componente a uma das cinco fases de aprendizagem, da teoria à prática supervisionada. Os percursos são criados a partir de *moldes*, o que evita reescrever estruturas comuns. Cada *supervisionado* recebe uma cópia própria do percurso, que o supervisor pode ajustar sem alterar o percurso dos restantes.

O supervisor avalia cada componente numa escala qualitativa de seis estados, e o sistema calcula automaticamente o progresso do estudante. Quando o estudante atinge uma meta, recebe uma *badge* e uma notificação.

1.3 Objetivos

Os objetivos deste trabalho são:

1. Analisar o processo de formação e avaliação e modelá-lo em Business Process Model and Notation (BPMN);
2. Definir os requisitos funcionais e não funcionais da aplicação;
3. Caracterizar o utilizador principal e modelar a sua tarefa principal;
4. Organizar os conteúdos e os ecrãs da aplicação num site map;
5. Documentar o sistema visual da aplicação;
6. Descrever a arquitetura, as rotas e os endpoints da Application Programming Interface (Interface de Programação de Aplicações) (API).

1.4 Âmbito

O trabalho cobre a aplicação web, nas suas três áreas de acesso: administração, supervisão e área do supervisionado. A aplicação é uma Single-Page Application (Aplicação de Página Única) (SPA) em Blazor WebAssembly, servida por uma API Representational State Transfer (REST) em ASP.NET Core sobre SQL Server. Ficam fora do âmbito a integração com sistemas académicos e a criação de contas pelos próprios estudantes.

[A completar: unidade curricular, docente e restantes elementos do grupo, se aplicável.]

1.5 Estrutura do documento

O Capítulo 2 descreve os processos de negócio em BPMN, os perfis de acesso e os requisitos. O Capítulo 3 caracteriza o utilizador principal e modela a sua tarefa. O Capítulo 4 apresenta o site map, e o Capítulo 5 o sistema visual. O Capítulo 6 descreve a arquitetura, as rotas e os endpoints. O Capítulo 7 conclui.

2. Análise do problema

Este capítulo descreve o contexto em que a aplicação é usada, os dois processos de negócio que ela suporta, os perfis de acesso e os requisitos.

2.1 Contexto e pressupostos

A aplicação destina-se a uma instituição de ensino que acompanha estudantes em formação perioperatória. Os pressupostos assumidos no desenho estão na Tabela 2.1.

Tema	**Pressuposto**
Contas	As contas são criadas pelo administrador; não há registo livre de estudantes.
Instituição	Uma instalação serve uma instituição; por omissão, a Universidade do Minho.
Fases	Cada componente pertence a uma de cinco fases: teórica, observação passiva, observação participada, prática assistida e prática supervisionada.
Avaliação	A avaliação é qualitativa, por componente, e só as fases práticas (assistida e supervisionada) contam para o progresso.
Planos	Cada estudante tem uma cópia própria do percurso, que pode divergir do percurso do grupo.

Tabela 2.1: Pressupostos assumidos no desenho.

2.2 Processos de negócio

Os processos estão modelados em BPMN (Object Management Group, 2014). Em cada diagrama, as tarefas a branco são feitas por uma pessoa e as tarefas a azul são feitas automaticamente pelo sistema.

2.2.1 Preparar um percurso e inscrever os supervisionados

Quando um supervisor recebe um novo grupo, cria um percurso a partir de um molde. Se não existir um molde adequado, o administrador cria-o primeiro. O sistema copia para o novo percurso os módulos, as componentes e as badges do molde, e o supervisor ajusta o resultado e as datas de cada fase (Figura 2.1).

Ao inscrever cada supervisionado, o sistema faz uma segunda cópia, desta vez do percurso para a *inscrição*. É esta cópia que o supervisor ajusta quando um estudante precisa de um plano diferente (modo *Individual* do construtor de percursos), sem alterar o percurso do grupo (modo *Geral*) nem o molde.

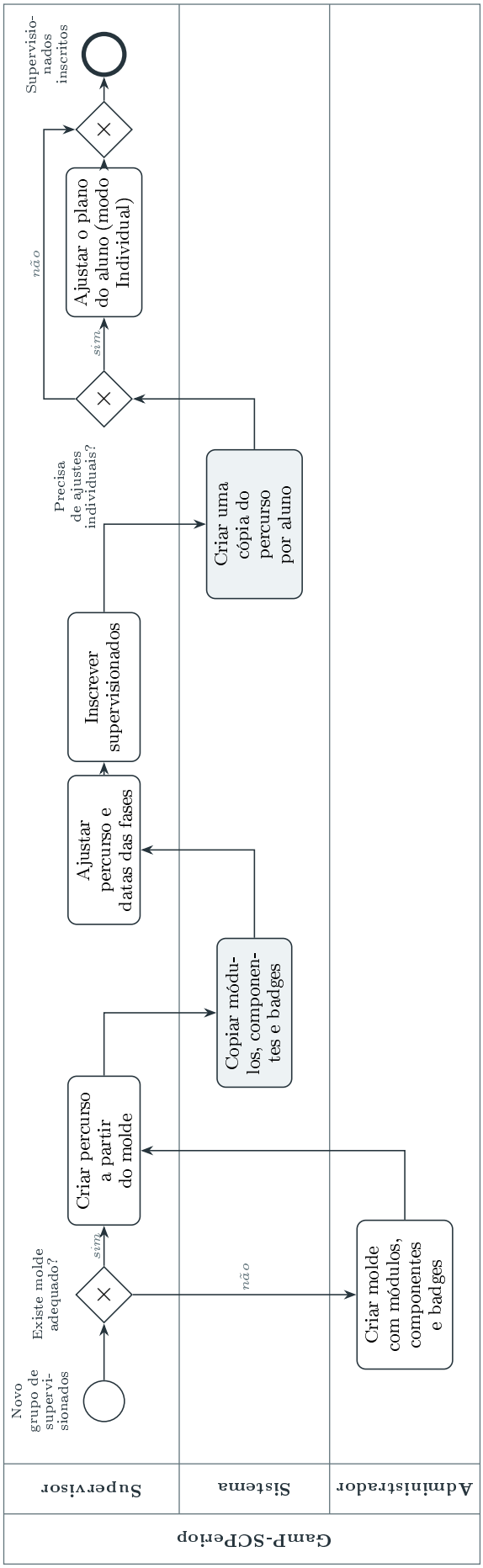


Figura 2.1: Preparar um percurso e inscrever os supervisionados, em BPMN.

2.2.2 Avaliar um supervisionado e atribuir badges

Depois de uma sessão prática, o supervisor abre a ficha de avaliação do estudante e classifica as componentes, módulo a módulo e fase a fase. Ao guardar, o sistema recalcula o progresso e verifica se o estudante ganhou alguma badge (Figura 2.2).

Cada componente recebe um de seis estados (Tabela 2.2). O progresso do percurso é a percentagem de componentes avaliáveis em *Acima da média* ou *Consistente*. São avaliáveis as componentes das fases práticas, com peso maior que zero, que não tenham subcomponentes. O mesmo cálculo é feito para cada módulo cuja avaliação mudou.

Estado	**Cor**	**Conta como concluída**
Pendente	Cinzento	Não
Em progresso	Azul	Não
Inconsistente	Vermelho	Não
Abaixo da média	Laranja	Não
Acima da média	Amarelo	Sim
Consistente	Verde	Sim

Tabela 2.2: Estados de avaliação de uma componente.

Há duas regras de atribuição de badges em funcionamento: uma badge de módulo quando o módulo atinge 65%, e uma badge de percurso quando o percurso atinge 100%. Cada badge é atribuída uma única vez a cada estudante. Em todos os casos, o estudante recebe uma notificação com uma ligação para a página correspondente.

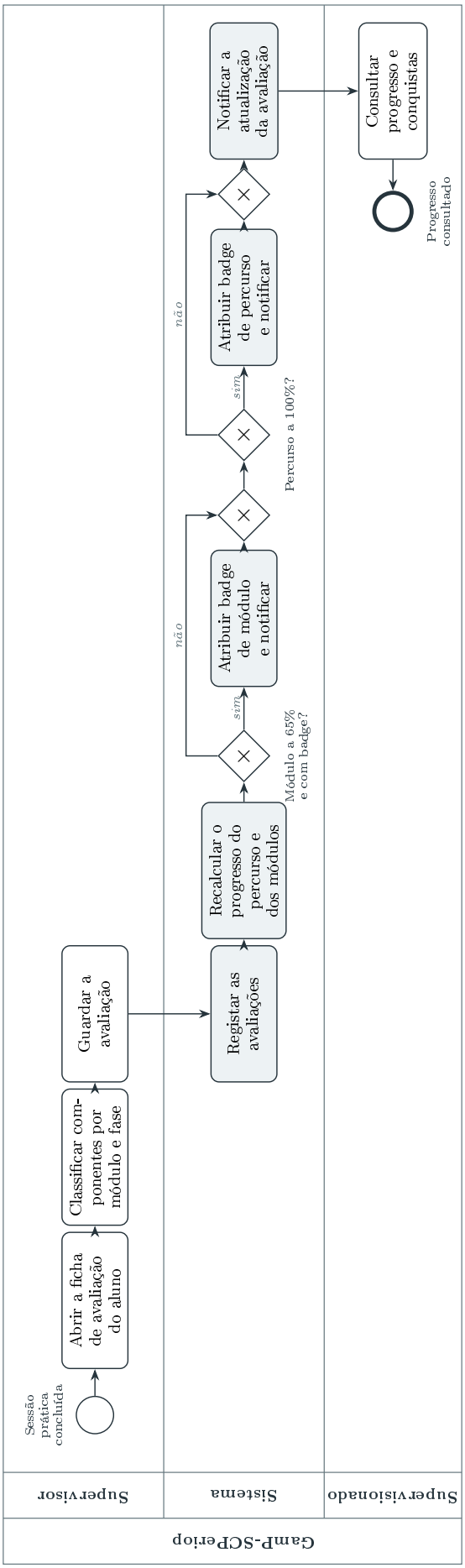


Figura 2.2: Avaliar um supervisionado e atribuir badges, em BPMN.

2.3 Atores e perfis de acesso

A aplicação tem três perfis, apresentados na Tabela 2.3. Cada utilizador tem um único perfil, e depois de entrar na aplicação é encaminhado para a página inicial desse perfil.

Perfil	O que pode fazer
Administrador	Gere as contas de supervisores e supervisionados, os moldes de percurso e as badges de cada molde.
Supervisor	Cria e ajusta os seus percursos, inscreve supervisionados, ajusta o plano de cada um e avalia as suas componentes.
Supervisionado	Consulta os seus percursos, módulos, componentes e documentos, o seu progresso e as suas badges; descarrega o relatório global de avaliação.

Tabela 2.3: Perfis de acesso.

2.4 Requisitos

Os requisitos estão agrupados por área funcional. Os requisitos não funcionais estão agrupados pelas categorias pedidas: usabilidade, acessibilidade, desempenho, segurança, compatibilidade e adaptação a diferentes dispositivos.

2.4.1 Requisitos funcionais

Id	Requisito
Autenticação e contas	
RF01	Autenticar o utilizador com email e palavra-passe e encaminhá-lo para a página inicial do seu perfil.
RF02	Permitir a qualquer utilizador alterar a sua palavra-passe.
RF03	Permitir ao administrador criar contas de supervisores e de supervisionados.
RF04	Permitir ao administrador repor a palavra-passe de um utilizador.
Moldes	
RF05	Gerir moldes de percurso: criar, consultar e apagar.
RF06	Gerir os módulos de um molde, com peso e ordem.
RF07	Gerir as componentes de um módulo do molde: título, descrição, fase, peso, ordem, documento Portable Document Format (PDF) e subcomponentes.
RF08	Gerir as badges de um molde: nome, descrição, ícone, cor, nível de raridade e regra de atribuição.
RF09	Carregar documentos PDF e associá-los a componentes.
Percursos e inscrições	
RF10	Criar um percurso vazio ou a partir de um molde, copiando os seus módulos, componentes e badges.
RF11	Editar um percurso: título, datas, módulos, componentes, pesos e ordem.
RF12	Definir as datas de início e fim de cada fase em cada módulo.

Id	Requisito
RF13	Apagar um percurso.
RF14	Inscrever um supervisionado num percurso, criando uma cópia própria dos módulos; impedir inscrições repetidas.
RF15	Ajustar o plano de um supervisionado sem alterar o percurso do grupo nem o molde.
RF16	Marcar inscrições como favoritas e ocultá-las do painel do supervisor.
Avaliação	
RF17	Classificar cada componente de um supervisionado com um dos seis estados da Tabela 2.2.
RF18	Recalcular o progresso do percurso e de cada módulo afetado sempre que uma avaliação é guardada.
RF19	Gerar o relatório global de avaliação de um supervisionado em PDF.
Gamificação e notificações	
RF20	Atribuir uma badge de módulo quando o módulo atinge 65% de progresso.
RF21	Atribuir uma badge de percurso quando o percurso atinge 100% de progresso.
RF22	Atribuir cada badge uma única vez a cada supervisionado, com a data em que foi obtida.
RF23	Notificar o supervisionado de cada nova avaliação e de cada nova badge, com uma ligação para a página correspondente.
RF24	Mostrar o número de notificações por ler e permitir marcá-las como lidas.
Consulta	
RF25	Mostrar ao supervisor os seus percursos, o número de supervisionados de cada um e o progresso de cada supervisionado.
RF26	Mostrar ao supervisionado os seus percursos, módulos, componentes, documentos e progresso.
RF27	Mostrar ao supervisionado as badges obtidas e as que ainda pode obter.

Tabela 2.4: Requisitos funcionais.

2.4.2 Requisitos não funcionais

Id	Requisito
Usabilidade	
RNF01	**Avaliação num só ecrã.** O supervisor avalia todas as componentes de um estudante num único ecrã, agrupadas por módulo e por fase.
RNF02	**Gravação explícita.** As classificações só são gravadas quando o supervisor confirma; o botão de gravar fica inativo enquanto não houver alterações.
RNF03	**Consistência.** Os estados de avaliação usam as mesmas cores e os mesmos nomes em todos os ecrãs.
RNF04	**Visibilidade do progresso.** O progresso é mostrado em barra e em percentagem no painel do supervisor e na área do supervisionado.
Acessibilidade	
RNF05	**Contraste.** O texto cumpre o contraste mínimo de 4,5:1 das Web Content Accessibility Guidelines (WCAG) 2.2, nível AA (World Wide Web Consortium, 2023), nos temas claro e escuro.

Id	Requisito
RNF06	**Cor não é a única pista.** Cada estado de avaliação é identificado por cor e por texto.
RNF07	**Teclado e leitores de ecrã.** Todas as ações podem ser feitas com o teclado, e os botões só com ícone têm um nome acessível.
Desempenho	
RNF08	**Navegação sem recarregar.** Depois do primeiro carregamento, a navegação entre ecrãs não recarrega a página.
RNF09	**Um pedido por gravação.** Guardar uma avaliação, recalcular o progresso e atribuir badges é feito num único pedido ao servidor.
Segurança	
RNF10	**Palavras-passe.** As palavras-passe são guardadas apenas como hash BCrypt e nunca são devolvidas pela API.
RNF11	**Autenticação.** Todos os pedidos à API, exceto a autenticação, exigem um JSON Web Token (JWT) válido, assinado pelo servidor e com validade limitada.
RNF12	**Autorização no servidor.** O servidor verifica o perfil e a posse dos dados em cada pedido: um supervisionado só acede aos seus dados e um supervisor só aos dos seus percursos.
RNF13	**Documentos protegidos.** Os documentos carregados só podem ser descarregados por utilizadores autenticados.
RNF14	**Dados pessoais.** Só são guardados os dados pessoais necessários (nome, email, instituição), em conformidade com o Regulamento Geral sobre a Proteção de Dados (RGPD).
Compatibilidade	
RNF15	**Navegadores.** A aplicação funciona nas versões atuais do Chrome, Edge, Firefox e Safari, que suportam WebAssembly.
RNF16	**Interoperabilidade.** A API REST está descrita em OpenAPI e pode ser explorada em `/swagger` em desenvolvimento.
Adaptação a diferentes dispositivos	
RNF17	**Layout adaptável.** Os ecrãs adaptam-se a telemóvel (menos de 576 px), tablet e computador (a partir de 1200 px), sem deslocamento horizontal da página.
RNF18	**Temas.** A aplicação oferece um tema claro e um tema escuro, escolhidos pelo utilizador.
RNF19	**Instalação.** A aplicação pode ser instalada como aplicação web progressiva, com ícone e nome próprios.

Tabela 2.5: Requisitos não funcionais.

3. Utilizador e task model principal

O utilizador principal é o supervisor, porque é ele quem cria os percursos, acompanha os estudantes e alimenta, com as suas avaliações, o progresso e as badges que os estudantes veem. Sem a sua atividade regular, a área do supervisionado fica vazia.

3.1 Utilizador principal

A Tabela 3.1 resume o perfil-tipo do supervisor.

Aspeto	**Descrição**
Quem é	Enfermeiro ou docente que orienta estudantes em contexto de bloco operatório, normalmente vários em simultâneo e em percursos diferentes.
Características	Domina a matéria clínica e os critérios de avaliação; tem competência digital média; tem pouco tempo entre turnos; usa o computador do serviço e, por vezes, um tablet ou telemóvel.
Necessidades	Montar planos sem os reescrever de raiz; ver de relance em que ponto está cada estudante; registar avaliações logo após a sessão prática, de forma rápida e sem ambiguidade.
Objetivos	Garantir que cada estudante passa por todas as fases de aprendizagem dentro dos prazos; ter um registo das avaliações que possa justificar; manter os estudantes motivados ao longo do percurso.
Frustrações atuais	Folhas de cálculo diferentes para cada grupo; progresso calculado à mão; avaliações em papel que se perdem ou ficam por registar.

Tabela 3.1: Perfil do utilizador principal: o supervisor.

O supervisionado é um utilizador secundário: consulta o seu plano e o seu progresso, mas não altera dados. O administrador usa a aplicação com pouca frequência, sobretudo no início de cada ano letivo.

3.2 Tarefa principal

A tarefa principal é **avaliar um supervisionado após uma sessão prática**. É a tarefa mais frequente do supervisor e é ela que faz avançar o progresso do estudante e que desencadeia a atribuição de badges (Figura 2.2).

A Figura 3.1 decompõe a tarefa numa análise hierárquica de tarefas (Hierarchical Task Analysis (Análise Hierárquica de Tarefas) (HTA)) (Annett & Stanton, 2000). Cada nível

divide a tarefa de cima em passos mais concretos, e os planos indicam a ordem em que esses passos são feitos.

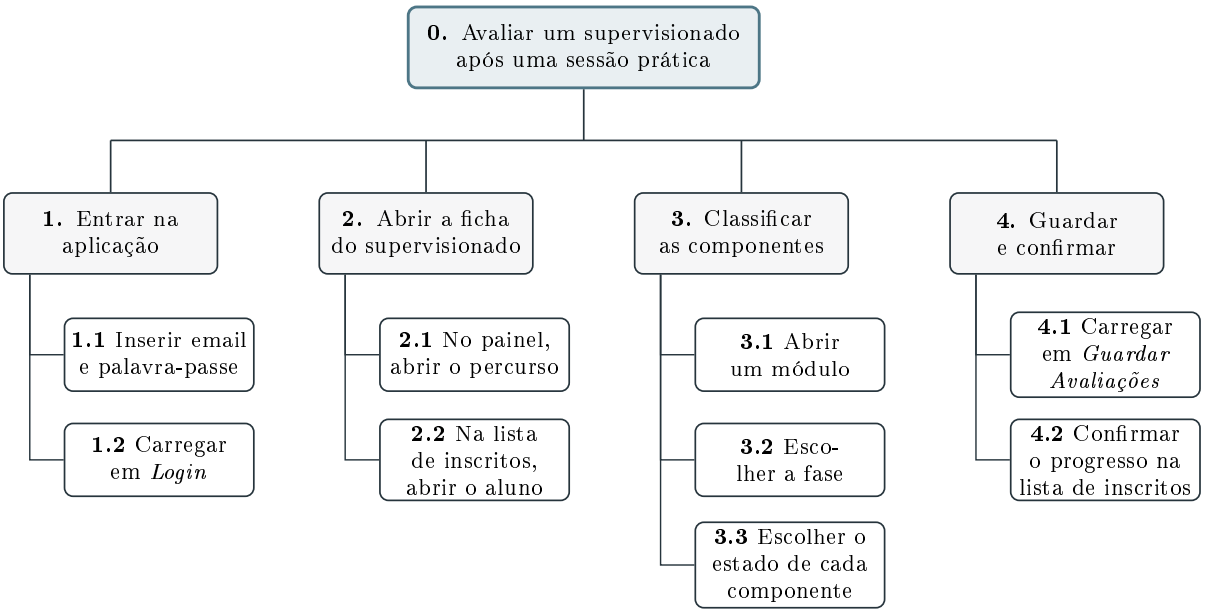


Figura 3.1: Task model da tarefa principal, em HTA.

A sequência de ações, a partir do ecrã de entrada, é a seguinte:

1. Na página de entrada, o supervisor insere o email e a palavra-passe e carrega em *Login*; a aplicação abre o painel do supervisor (`/dashboard`).
2. No painel, carrega no cartão do percurso; abre-se a gestão do percurso, com a lista de supervisionados inscritos (`/pathway/{id}/students`).
3. Na lista, carrega no estudante; abre-se a ficha de avaliação (`/evaluate/{id}`).
4. Na ficha, abre um módulo, escolhe a fase prática e, para cada componente, carrega num dos estados: *Inconsistente*, *Abaixo da média*, *Acima da média* ou *Consistente*.
5. Repete o passo anterior para cada fase e cada módulo que avaliou na sessão.
6. Carrega em *Guardar Avaliações*. O sistema grava as classificações, recalcula o progresso, atribui as badges devidas e notifica o estudante.
7. Volta à lista de inscritos e confirma o novo valor de *Progresso no Percurso*.

Sem contar com a autenticação, a tarefa exige três ecrãs e duas navegações até à ficha de avaliação. O resto do trabalho acontece num único ecrã, como pede o requisito RNF01.

4. Site map

A aplicação divide-se numa área pública, com a entrada, e numa área autenticada com uma secção por perfil. Depois do login, cada utilizador é levado para a página inicial do seu perfil e só vê os ecrãs dessa secção, mais os que são comuns a todos (Figura 4.1).

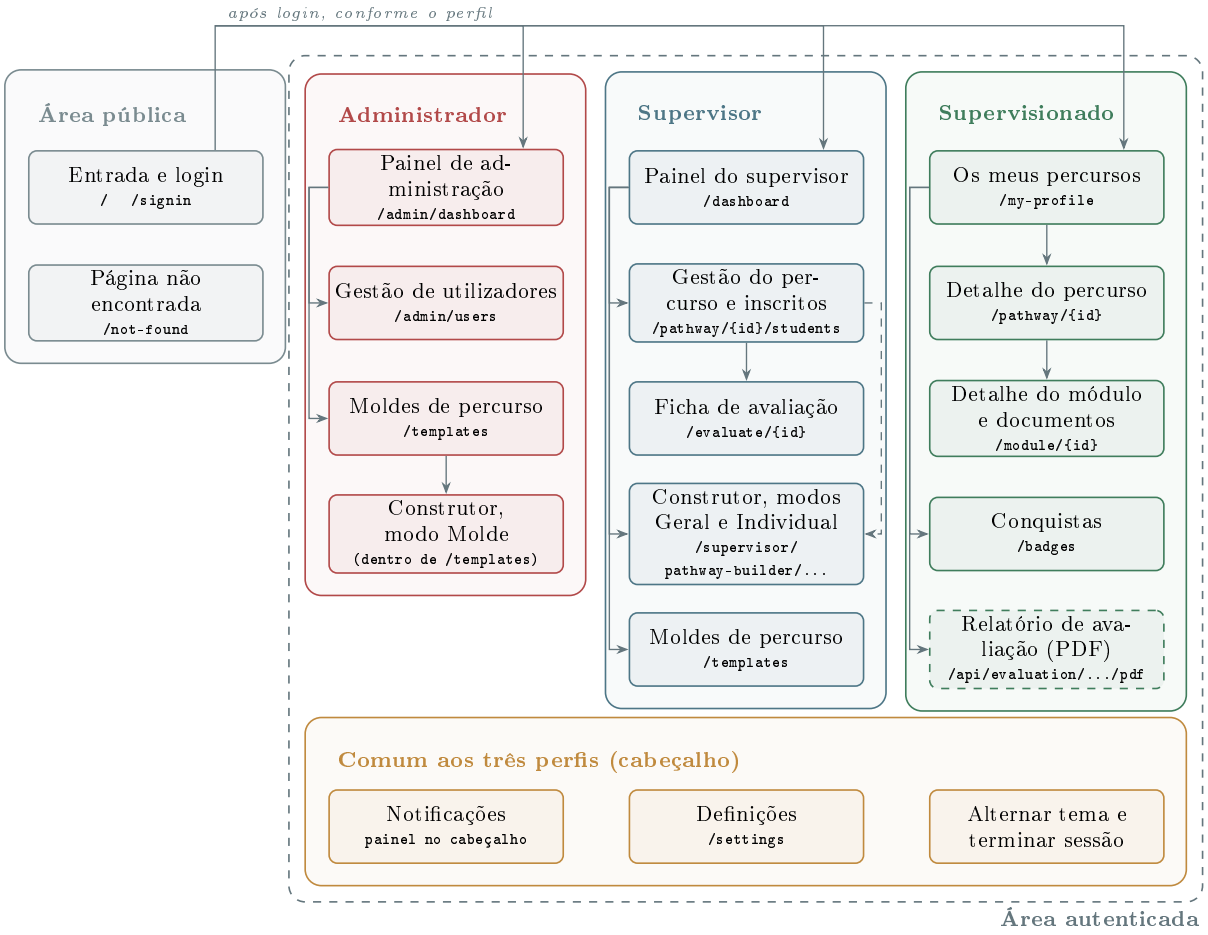


Figura 4.1: Site map do GamP-SCPeriop. Cada caixa é um ecrã, com a rota por baixo; as setas cheias são a navegação principal e a tracejada um atalho.

4.1 Organização dos conteúdos

Os conteúdos seguem a hierarquia do domínio: um percurso tem módulos, e um módulo tem componentes. O site map reproduz essa hierarquia nas duas áreas que a mostram:

- Na área do supervisor, o painel lista os percursos; cada percurso abre a lista de inscritos, e cada inscrito abre a sua ficha de avaliação.

- Na área do supervisionado, *Os meus percursos* lista os percursos; cada percurso abre os seus módulos, e cada módulo as suas componentes e documentos.

4.2 Áreas e diferenças de acesso

A Tabela 4.1 indica, para cada área, quem lhe pode aceder e por onde se entra.

Área	**Acesso**	**Entrada e ecrãs principais**
Pública	Qualquer visitante	Entrada e login (/); página não encontrada.
Administrador	Perfil Administrador	Painel de administração; gestão de utilizadores; moldes de percurso e respetivo construtor.
Supervisor	Perfil Supervisor	Painel com os percursos; gestão de cada percurso e dos inscritos; ficha de avaliação; construtor de percursos; moldes.
Supervisionado	Perfil Supervisionado	Os meus percursos; detalhe do percurso e do módulo; conquistas; relatório de avaliação em PDF.
Comum	Os três perfis	Notificações e alternância de tema no cabeçalho; definições (alterar a palavra-passe); terminar sessão.

Tabela 4.1: Áreas da aplicação e respetivo acesso.

4.3 Ligações entre ecrãs

Além da navegação hierárquica, há três tipos de ligação que atravessam o site map:

- **Construtor de percursos.** É o mesmo ecrã em três modos. O administrador usa-o em modo *Molde*; o supervisor usa-o em modo *Geral*, a partir do painel ou da gestão do percurso, e em modo *Individual*, a partir de um inscrito.
- **Notificações.** Cada notificação leva diretamente ao ecrã a que diz respeito: uma nova avaliação abre o detalhe do percurso, e uma nova badge abre as conquistas.
- **Moldes.** O ecrã de moldes é partilhado pelo administrador, que gere os moldes-base, e pelo supervisor, que os consulta para criar percursos.

Os ecrãs abaixo da página inicial de cada perfil têm um botão para voltar ao ecrã de cima, e o cabeçalho é igual em todos os ecrãs autenticados.

5. Sistema visual

O sistema visual parte de um azul-acinzentado calmo, adequado a um contexto clínico, e reserva as cores fortes para comunicar o estado da avaliação. Todos os valores deste capítulo foram retirados da folha de estilos da aplicação (`wwwroot/css/app.css`).

5.1 Logótipo e identidade visual

A aplicação ainda não tem logótipo. O cabeçalho mostra o nome da instituição (*Universidade do Minho*) em texto, e os ícones da aplicação instalável são os ícones por omissão do Blazor.

[A completar: logótipo ou símbolo do GamP-SCPeriop, versões para fundo claro e escuro, área de proteção e tamanho mínimo.]

5.2 Paleta cromática

A paleta está definida como variáveis CSS, com um valor para o tema claro e outro para o tema escuro (Tabela 5.1). A mudança de tema troca os valores das variáveis, sem alterar os ecrãs.

Variável	Claro	Hex claro	Hex escuro	Função
`--brand`		#6A96A6	#8FB6C4	Cor da marca: ícones do cabeçalho, barras de progresso *em curso*, ligações no tema escuro.
`--brand-dark`		#4D7686	#6A96A6	Texto do estado *em curso* e percentagens de progresso.
`--ink`		#26343C	#E7E8E9	Texto principal e títulos.
`--ink-soft`		#64767D	#9DA0A3	Texto secundário, legendas e metadados.
`--paper`		#E9EAEB	#1B1C1E	Fundo da página.
`--card-bg`		#F6F6F7	#28292B	Fundo dos cartões.
`--line`		#DADCDD	#3A3B3D	Contornos, divisórias e trilho das barras de progresso.
`--approved`		#3F7D5C	#6FB98D	Estado positivo: concluído, aprovado, consistente.
`--reproved`		#B24949	#D97A7A	Estado negativo: reprovado, inconsistente.
`--pendente`		#C08A3E	#E0B36E	Estado intermédio: em curso, a aguardar.
`--por-iniciar`		#7A8B90	#7A8B90	Estado neutro: ainda não iniciado.
`--star`		#D9A441	#D9A441	Inscrições favoritas (estrela).

Tabela 5.1: Paleta cromática da aplicação, com a função de cada cor.

Cada cor de estado tem uma variante de fundo (`--approved-bg`, `--reproved-bg`, `--pendente-bg` e `--por-iniciar-bg`), usada nas etiquetas de estado: o texto leva a cor do estado e o fundo a sua variante clara.

Há três desvios à paleta na implementação atual: os botões principais usam o azul #1B6EC2 do modelo-base do Blazor, que não pertence à paleta; os ecrãs ainda têm 74 códigos de cor escritos diretamente, que não mudam com o tema; e `--em-curso-bg` só está definida no tema escuro. Propõe-se usar `--brand-dark` nos botões principais e substituir os códigos diretos pelas variáveis.

5.2.1 Função das cores principais

- **Marca** (`--brand`, `--brand-dark`): identifica o que pertence à aplicação ou está em curso, como os ícones do cabeçalho, as ligações e as barras de progresso.
- **Neutros** (`--ink`, `--paper`, `--card-bg`, `--line`): estruturam a página. O fundo é ligeiramente mais escuro que os cartões, para que estes se destaquem sem precisarem de contornos fortes.
- **Estados** (verde, vermelho, âmbar, cinzento): comunicam o resultado de uma avaliação ou o estado de um percurso. São as únicas cores saturadas da interface, para que um problema se veja de relance.

5.2.2 Contraste

A Tabela 5.2 mostra o contraste das combinações mais usadas, calculado segundo as WCAG 2.2 (World Wide Web Consortium, 2023). O requisito RNF05 exige pelo menos 4,5:1 para texto.

Texto sobre fundo	Claro	Escuro	Cumpre 4,5:1
`--ink` sobre `--card-bg`	11,86	11,87	Sim
`--ink-soft` sobre `--card-bg`	4,39	5,54	Só no escuro
`--brand-dark` sobre `--card-bg`	4,57	4,52	Sim
Branco sobre o botão principal (#1B6EC2)	5,18	5,18	Sim
`--approved` sobre `--approved-bg`	4,16	5,23	Só no escuro
`--reproved` sobre `--reproved-bg`	4,43	4,86	Só no escuro
`--pendente` sobre `--pendente-bg`	2,53	6,83	Só no escuro
`--por-iniciar` sobre `--por-iniciar-bg`	3,03	3,03	Não

Tabela 5.2: Contraste das principais combinações de cor.

No tema claro, as etiquetas de estado e o texto secundário ficam abaixo de 4,5:1. A correção mais simples é escurecer a cor do texto de cada estado (por exemplo, `--pendente` para um âmbar mais escuro) mantendo os fundos. No tema escuro, `--por-iniciar-bg` mantém o valor claro, que deve ser substituído por um cinzento escuro.

5.3 Tipografia

A aplicação usa uma única família, a *Plus Jakarta Sans* (Tokotype, 2020), carregada do Google Fonts nos pesos 400, 500, 600 e 700. É uma letra sem serifa, geométrica e muito legível em tamanhos pequenos, o que importa em ecrãs densos como a ficha de avaliação.

Uso	Peso	Onde
Títulos de página e de secção	700 (bold)	Nome do percurso, títulos dos ecrãs.
Nomes e rótulos	600 (semibold)	Nomes de módulos e componentes, botões, etiquetas de estado (0,72 rem).
Texto corrente	400 (regular)	Descrições, listas e formulários.
Texto secundário	400, cor `--ink-soft`	Datas, legendas e metadados.

Tabela 5.3: Utilização da tipografia.

Os ícones são da biblioteca *Bootstrap Icons* 1.11.3, em traço simples, coerentes com o peso da letra. As badges usam o mesmo conjunto de ícones (por omissão, `bi-trophy`).

5.4 Grelha, margens e espaçamento

A disposição assenta na grelha de 12 colunas do Bootstrap 5.3 (Bootstrap Team, 2024), dentro de um contentor centrado cuja largura e margens variam com o ecrã (Tabela 5.4).

Ecrã	Largura máxima	Margem interior	Notas
Telemóvel ($<$ 576 px)	100%	16 px (1 rem)	Letra base reduzida para 14 px (0,875 rem).
Tablet e portátil (576–1199 px)	960 px	24 px (1,5 rem)	
Computador ($\geq$ 1200 px)	1400 px	32 px (2 rem)	

Tabela 5.4: Contentor principal por tamanho de ecrã.

Os princípios de espaçamento são:

- **Escala de 4 px.** Os espaçamentos usam a escala do Bootstrap: 4, 8, 16, 24 e 48 px.
- **Cartões como unidade.** Cada percurso, aluno ou módulo é um cartão com cantos de 16 px, sombra leve e sem contorno. Ao passar o rato, o cartão sobe 5 px para indicar que é clicável.
- **Formas arredondadas.** Etiquetas de estado em forma de pílula (raio 20 px) e barras de progresso com 8 px de altura e cantos arredondados.

Os raios dos cantos variam hoje entre 8, 10, 12, 15, 16 e 20 px de ecrã para ecrã. Propõe-se reduzi-los a três valores: 8 px para botões e campos, 16 px para cartões, e forma de pílula para etiquetas.

6. Arquitetura

O GamP-SCPeriop é um monólito: um único programa ASP.NET Core serve a aplicação cliente e a API, e guarda tudo numa única base de dados. Este capítulo justifica essa escolha, descreve os componentes, explica como são resolvidas as rotas do cliente e da API, e apresenta o catálogo completo de endpoints.

6.1 Estilo arquitetural

Escolheu-se um monólito com três projetos, em vez de serviços independentes, pelas seguintes razões:

- **Escala.** A aplicação serve uma instituição, com dezenas de supervisores e algumas centenas de estudantes. Não há partes do sistema com cargas tão diferentes que justifiquem escalá-las em separado.
- **Consistência.** As operações principais tocam muitas tabelas de uma só vez: criar um percurso a partir de um molde copia módulos, componentes e badges, e guardar uma avaliação atualiza o progresso e atribui badges. Numa única base de dados, estas operações são transações simples.
- **Equipa e operação.** Um único programa, desenvolvido por uma equipa pequena, é mais simples de testar, de pôr em produção e de manter.
- **Código partilhado.** Cliente e servidor usam as mesmas classes de entidades e de DTOs, definidas uma única vez na biblioteca partilhada.

O custo desta escolha é que todas as partes são publicadas em conjunto e que uma falha no servidor afeta toda a aplicação. Para o contexto em causa, esse custo é aceitável.

6.2 Componentes

A solução tem três projetos (Tabela 6.1). O servidor aloja o cliente: no primeiro acesso, o navegador descarrega a aplicação Blazor WebAssembly como ficheiros estáticos; a partir daí, a aplicação corre no navegador e só comunica com o servidor através da API REST (Figura 6.1).

Projeto	Responsabilidade
`GamP-SCPeriop`	Cliente Blazor WebAssembly: páginas, layouts, componentes reutilizáveis, estado de autenticação e chamadas à API.
`GamP-SCPeriop.Server`	Servidor ASP.NET Core: aloja o cliente, expõe a API REST, valida os JWT, acede à base de dados com Entity Framework Core e gera os relatórios PDF com QuestPDF.
`GamP-SCPeriop.Shared`	Biblioteca partilhada: entidades, DTOs e enumerações usados pelos dois lados.

Tabela 6.1: Projetos da solução.

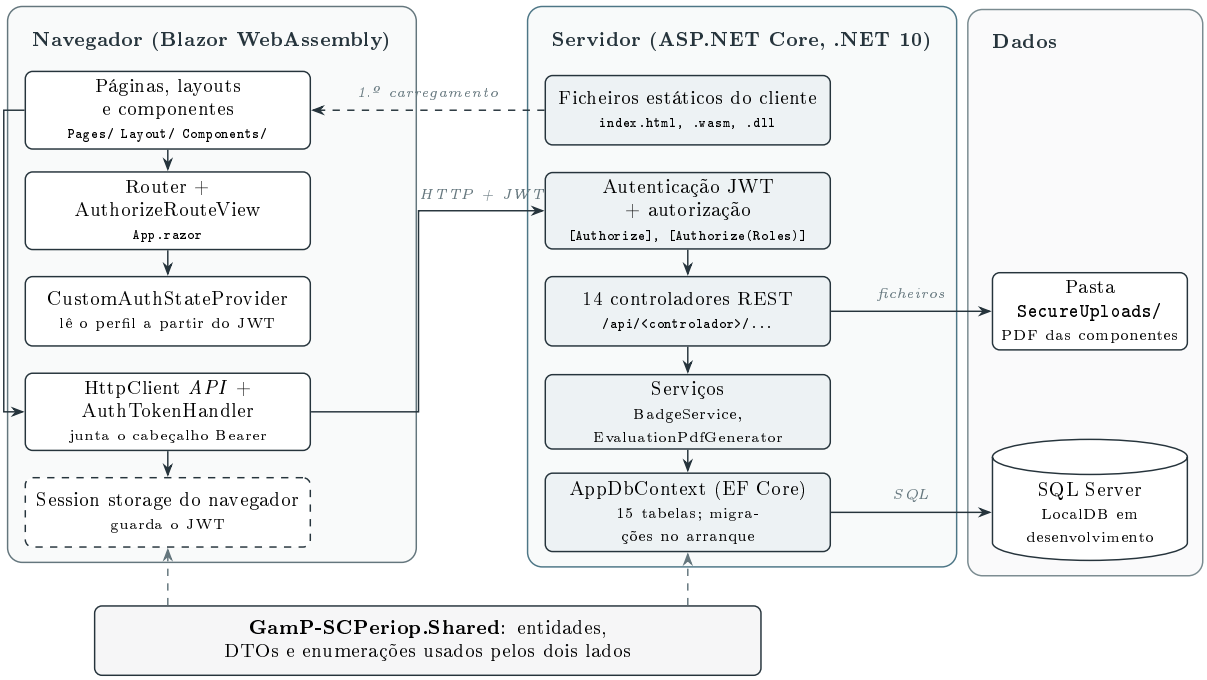


Figura 6.1: Arquitetura do GamP-SCPeriop. A azul, os componentes do servidor; a seta tracejada é o descarregamento inicial da aplicação.

6.2.1 Pipeline do servidor

Cada pedido atravessa, por esta ordem, os componentes configurados em `Program.cs`:

1. **CORS**, que aceita pedidos de qualquer origem;
2. **Redireção para HTTPS**;
3. **Ficheiros estáticos**, incluindo os ficheiros da aplicação Blazor (`UseBlazorFrameworkFiles` e `UseStaticFiles`);
4. **Encaminhamento**, que escolhe o controlador e a ação;
5. **Autenticação e autorização**, que validam o JWT e os atributos `[Authorize]`;

6. **Controladores**, e por fim o **fallback**, que devolve `index.html` a qualquer pedido que não corresponda a um ficheiro nem a um controlador.

No arranque, o servidor aplica as migrações pendentes à base de dados (32 até à data) e, se não existir nenhum utilizador, cria a conta de administrador inicial. Em desenvolvimento, publica também a documentação OpenAPI em `/swagger`.

6.2.2 Serviços e lógica de negócio

A lógica de negócio está repartida entre os controladores e dois serviços:

- `BadgeService`: verifica e atribui as badges de módulo e de percurso, e cria as notificações correspondentes;
- `EvaluationPdfGenerator`: gera o relatório global de avaliação de um estudante.

As cópias de moldes e de percursos e o cálculo do progresso estão escritos diretamente nos controladores (`PathwayController`, `EnrollmentController` e `EvaluationController`). Mover esta lógica para serviços próprios tornaria o código mais fácil de testar, e é a primeira melhoria proposta para a arquitetura.

6.3 Rotas

A aplicação tem dois conjuntos de rotas, resolvidos em lugares diferentes:

- **Rotas do cliente**, como `/dashboard` ou `/pathway/7`. São declaradas em cada página com a diretiva `@page` e resolvidas no navegador pelo *Router* do Blazor. O Capítulo 4 lista-as todas.
- **Rotas da API**, todas começadas por `/api/`. São declaradas com atributos nos controladores e resolvidas no servidor.

A Figura 6.2 mostra o percurso de um pedido. O servidor tenta, por ordem, servir um ficheiro estático e encontrar um controlador; se nenhum corresponder, devolve `index.html`, e a aplicação Blazor, já no navegador, escolhe a página.

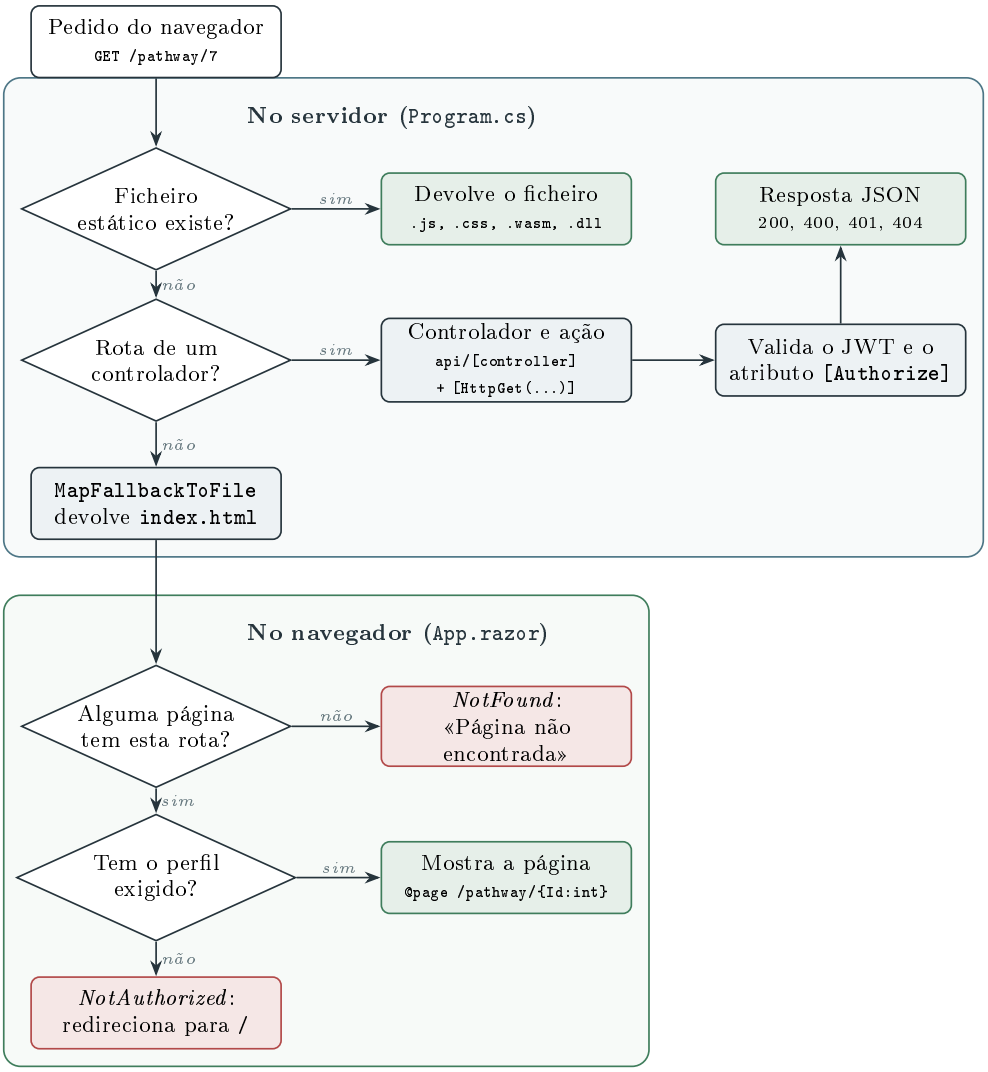


Figura 6.2: Resolução de um URL: primeiro no servidor, depois no navegador.

6.3.1 Rotas do cliente

Cada página declara a sua rota no topo do ficheiro com a diretiva `@page`, por exemplo `@page "/pathway/{Id:int}"`. As restrições de tipo (`:int`) fazem com que `/pathway/abc` não corresponda à página. Uma página pode declarar várias rotas: o construtor de percursos responde a três, uma por modo.

O *Router* está envolvido num `AuthorizeRouteView`. Se a página tiver um atributo `[Authorize(Roles = ...)]` e o utilizador não tiver esse perfil, a aplicação redireciona-o para a página de entrada. O perfil é lido do JWT guardado no navegador, pelo `CustomAuthStateProvider`.

Como o servidor devolve `index.html` a qualquer rota desconhecida, é possível abrir diretamente uma rota do cliente, por exemplo a partir de uma notificação ou de um marcador.

6.3.2 Rotas da API

Todos os controladores usam o mesmo modelo de rota,

`[Route("api/[controller]")]`,

em que `[controller]` é substituído pelo nome da classe sem o sufixo `Controller`. Cada ação acrescenta o seu segmento com o atributo do método HTTP. Por exemplo:

```
[Route("api/[controller]")]         // -> api/pathway
public class PathwayController : ControllerBase
{
    [HttpGet("supervisor/{supervisorId}")]
    // GET api/pathway/supervisor/5
    public async Task<...> GetSupervisorPathways(int supervisorId)
}
```

Listagem 6.1: Rota de uma ação em `PathwayController`.

As regras que se aplicam a toda a API são:

- As rotas não distinguem maiúsculas de minúsculas: `/api/Pathway` e `/api/pathway` são a mesma.
- Os segmentos entre chavetas (`{id}`) passam para o parâmetro com o mesmo nome; os corpos dos pedidos chegam em JSON (`[FromBody]` ou DTO).
- As respostas usam os códigos habituais: 200 com JSON, 204 sem conteúdo, 400 para pedidos inválidos, 401 sem JWT válido e 404 quando o recurso não existe.
- A serialização ignora referências circulares (`ReferenceHandler.IgnoreCycles`), porque as entidades se referem umas às outras.

6.4 Autenticação e autorização

A autenticação usa JWT assinados pelo próprio servidor:

1. `POST /api/auth/login` verifica o email e o hash BCrypt da palavra-passe e devolve um token com o identificador, o nome, o email e o perfil do utilizador, válido durante 7 dias.
2. O cliente guarda o token na *session storage* do navegador. Por isso, a sessão termina quando o separador é fechado.
3. O `AuthTokenHandler` junta o cabeçalho `Authorization: Bearer` a todos os pedidos feitos pelo `HttpClient` da aplicação.
4. O servidor valida a assinatura, o emissor, o destinatário e a validade de cada token.

A Figura 6.3 mostra este mecanismo num pedido real: gravar uma avaliação.

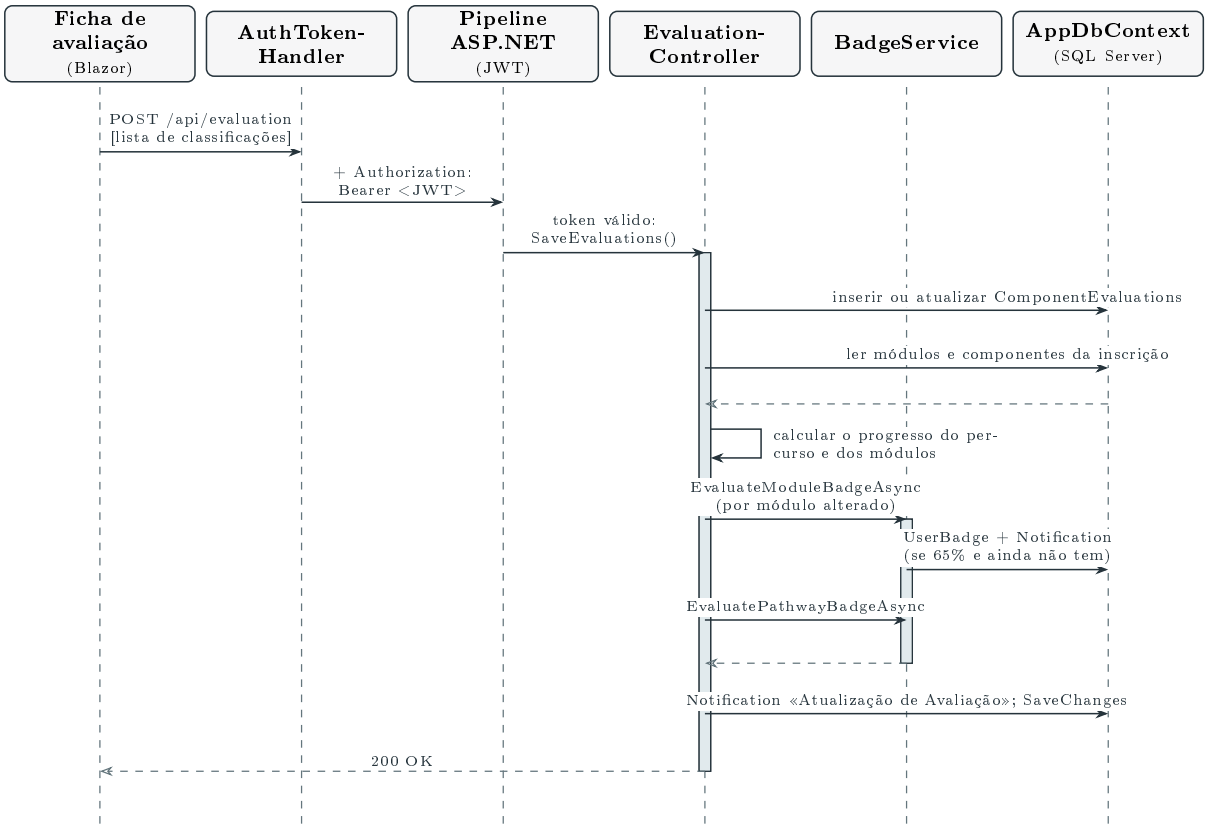


Figura 6.3: Sequência de pedidos ao gravar uma avaliação.

Todos os controladores exigem um utilizador autenticado (`[Authorize]`), com três exceções: o registo, o login e a transferência de ficheiros. A verificação do perfil, porém, só existe em dois endpoints. Nos restantes, a separação entre perfis é feita apenas pelas páginas do cliente, e os identificadores de utilizador chegam no URL em vez de serem lidos do token. Cumprir o requisito RNF12 exige verificar o perfil e a posse dos dados no servidor, em cada pedido.

6.5 Catálogo de endpoints

A API tem 54 endpoints, distribuídos por 14 controladores (Tabela 6.2). A Tabela 6.3 lista-os todos. Na coluna *Acesso*, *Autenticado* significa qualquer utilizador com um JWT válido.

Controlador	Endpoints	Responsabilidade
`Auth`	4	Registo, login e palavras-passe.
`User`	2	Listagem de utilizadores.
`PathwayTemplate`	4	Moldes de percurso.
`ModuleTemplate`	4	Módulos dos moldes.
`ComponentTemplate`	4	Componentes dos moldes.
`BadgeTemplate`	4	Badges dos moldes.
`Pathway`	7	Percursos e cópia a partir de moldes.
`Module`	6	Módulos e datas das fases.
`ModuleComponent`	4	Componentes dos módulos.
`Enrollment`	7	Inscrições e cópia do percurso por aluno.
`Evaluation`	3	Avaliações, progresso e relatório PDF.
`Badges`	1	Badges obtidas por um supervisionado.
`Notification`	2	Notificações.
`Files`	2	Carregamento e transferência de documentos.
Total	**54**	

Tabela 6.2: Controladores da API.

Método	Rota	Descrição	Acesso
AuthController — `/api/auth`			
`POST`	`/api/auth/register`	Criar utilizador	Público
`POST`	`/api/auth/login`	Autenticar e obter o JWT	Público
`POST`	`/api/auth/change-password`	Alterar a própria palavra-passe	Autenticado
`POST`	`/api/auth/reset-password`	Repor a palavra-passe de um utilizador	Admin
UserController — `/api/user`			
`GET`	`/api/user`	Utilizadores que não são administradores	Autenticado
`GET`	`/api/user/students`	Supervisionados	Autenticado
PathwayTemplateController — `/api/pathwaytemplate`			
`GET`	`/api/pathwaytemplate`	Listar moldes	Autenticado
`GET`	`/api/pathwaytemplate/{id}`	Obter um molde	Autenticado
`POST`	`/api/pathwaytemplate`	Criar molde	Autenticado
`DELETE`	`/api/pathwaytemplate/{id}`	Apagar molde	Autenticado
ModuleTemplateController — `/api/moduletemplate`			
`GET`	`/api/moduletemplate/{id}`	Obter módulo do molde	Autenticado
`POST`	`/api/moduletemplate`	Criar módulo do molde	Autenticado
`PUT`	`/api/moduletemplate/{id}`	Atualizar módulo do molde	Autenticado
`DELETE`	`/api/moduletemplate/{id}`	Apagar módulo do molde	Autenticado
ComponentTemplateController — `/api/componenttemplate`			
`GET`	`/api/componenttemplate/{id}`	Obter componente do molde	Autenticado
`POST`	`/api/componenttemplate`	Criar componente do molde	Autenticado
`PUT`	`/api/componenttemplate/{id}`	Atualizar componente do molde	Autenticado
`DELETE`	`/api/componenttemplate/{id}`	Apagar componente do molde	Autenticado

Método	Rota	Descrição	Acesso
BadgeTemplateController — `/api/badgetemplate`			
`GET`	`/api/badgetemplate/pathway/{pathwayTemplateId}`	Badges de um molde	Autenticado
`POST`	`/api/badgetemplate`	Criar badge do molde	Autenticado
`PUT`	`/api/badgetemplate/{id}`	Atualizar badge do molde	Autenticado
`DELETE`	`/api/badgetemplate/{id}`	Apagar badge do molde	Autenticado
PathwayController — `/api/pathway`			
`POST`	`/api/pathway`	Criar percurso (copia o molde, se indicado)	Autenticado
`GET`	`/api/pathway/{id}`	Obter percurso	Autenticado
`GET`	`/api/pathway/supervisor/{supervisorId}`	Percursos de um supervisor	Autenticado
`GET`	`/api/pathway/{pathwayId}/enrollments`	Inscrições de um percurso	Autenticado
`GET`	`/api/pathway/builder/{id}`	Cópia de um aluno, para o construtor	Autenticado
`PUT`	`/api/pathway/{id}/title`	Mudar o título	Autenticado
`DELETE`	`/api/pathway/{id}`	Apagar percurso	Autenticado
ModuleController — `/api/module`			
`POST`	`/api/module`	Criar módulo	Autenticado
`GET`	`/api/module/{id}`	Obter módulo	Autenticado
`GET`	`/api/module/{moduleId}/student/{studentId}`	Módulo com as avaliações de um aluno	Autenticado
`PUT`	`/api/module/{id}`	Atualizar módulo	Autenticado
`PUT`	`/api/module/{id}/timeline`	Atualizar as datas das fases de um aluno	Autenticado
`DELETE`	`/api/module/{id}`	Apagar módulo	Autenticado
ModuleComponentController — `/api/modulecomponent`			
`POST`	`/api/modulecomponent`	Criar componente	Autenticado
`GET`	`/api/modulecomponent/{id}`	Obter componente	Autenticado
`PUT`	`/api/modulecomponent/{id}`	Atualizar componente	Autenticado
`DELETE`	`/api/modulecomponent/{id}`	Apagar componente	Autenticado
EnrollmentController — `/api/enrollment`			
`POST`	`/api/enrollment`	Inscrever aluno (copia o percurso)	Autenticado
`GET`	`/api/enrollment/supervisor/{supervisorId}`	Inscrições dos percursos de um supervisor	Autenticado
`GET`	`/api/enrollment/student/{studentId}`	Inscrições de um aluno	Autenticado
`GET`	`/api/enrollment/student/{studentId}/pathway/{pathwayId}`	Detalhe de uma inscrição	Autenticado
`GET`	`/api/enrollment/management`	Alunos, para a gestão de utilizadores	Supervisor, Admin
`PUT`	`/api/enrollment/{id}/star`	Marcar ou desmarcar como favorita	Autenticado
`PUT`	`/api/enrollment/{id}/hidden`	Ocultar ou mostrar no painel	Autenticado
EvaluationController — `/api/evaluation`			
`GET`	`/api/evaluation/enrollment/{id}`	Inscrição com as avaliações	Autenticado
`POST`	`/api/evaluation`	Gravar avaliações; recalcular progresso e badges	Autenticado

Método	Rota	Descrição	Acesso
GET	/api/evaluation/student/{studentId}/pdf	Relatório global em PDF	Autenticado
BadgesController — /api/badges			
GET	/api/badges/student/{studentId}	Badges de um aluno	Autenticado
NotificationController — /api/notification			
GET	/api/notification/user/{userId}	Notificações de um utilizador	Autenticado
PUT	/api/notification/{id}/read	Marcar como lida	Autenticado
FilesController — /api/files			
POST	/api/files/upload	Carregar um documento	Autenticado
GET	/api/files/download/{fileName}	Transferir um documento	Público

Tabela 6.3: Catálogo de endpoints da API.

6.6 Dados

A base de dados é gerida com Entity Framework Core em modo *code-first*: as tabelas são geradas a partir das classes da biblioteca partilhada e evoluem através de migrações. São 15 tabelas, organizadas em três camadas que correspondem às duas cópias descritas na Secção 6.1: moldes, percursos partilhados e cópias por aluno. As chaves estrangeiras entre inscrições, avaliações e utilizadores usam `Restrict`, para que apagar um utilizador ou um módulo não apague em cascata o histórico de avaliações.

Os documentos PDF carregados são guardados fora da pasta pública do servidor, em `SecureUploads/`, com um nome aleatório (GUID), e só são acessíveis através do endpoint de transferência.

[A completar: diagrama entidade-relação das 15 tabelas, se o orientador o pedir.]

7. Conclusão

Este documento modelou os dois processos centrais do GamP-SCPeriop, a preparação de percursos e a avaliação com atribuição de badges, e fixou 27 requisitos funcionais e 19 não funcionais. Identificou o supervisor como utilizador principal, decompôs a sua tarefa principal e organizou os ecrãs num site map com uma área por perfil. Documentou o sistema visual a partir da folha de estilos da aplicação e descreveu a arquitetura: um monólito com um cliente Blazor WebAssembly e uma API REST de 54 endpoints.

A ideia que organiza a aplicação é a dupla cópia: do molde para o percurso, e do percurso para cada estudante. É ela que permite reutilizar planos e, ao mesmo tempo, adaptar o plano de cada estudante sem afetar os outros.

A comparação com a implementação atual deixa trabalho identificado:

- **Segurança (RNF10, RNF12, RNF13).** O registo de contas está aberto sem autenticação e aceita o perfil indicado no pedido; a maioria dos endpoints só verifica que o utilizador entrou, e não o seu perfil nem a posse dos dados; a transferência de documentos não exige autenticação.
- **Arquitetura.** A lógica de cópia e de cálculo do progresso está nos controladores e deve passar para serviços, para poder ser testada isoladamente.
- **Acessibilidade (RNF05).** As etiquetas de estado e o texto secundário não atingem 4,5:1 de contraste no tema claro.
- **Avaliação.** Os pesos e as notas mínimas de aprovação são guardados mas ainda não entram no cálculo do progresso, e duas das quatro regras de badges (excelência e rapidez) estão definidas mas não são aplicadas.
- **Identidade.** Falta um logótipo, e os raios dos cantos devem ser uniformizados.

Bibliografia

Annett, John & Neville A. Stanton. 2000. *Task analysis*. London: Taylor & Francis.

Bootstrap Team. 2024. Bootstrap 5 – grid system. https://getbootstrap.com/docs/5.3/layout/grid/.

Object Management Group. 2014. Business process model and notation (bpmn), version 2.0.2. https://www.omg.org/spec/BPMN/2.0.2/.

Tokotype. 2020. Plus jakarta sans. https://fonts.google.com/specimen/Plus+Jakarta+Sans.

World Wide Web Consortium. 2023. Web content accessibility guidelines (wcag) 2.2. https://www.w3.org/TR/WCAG22/.